



Fachhochschule Aachen, Campus Jülich

Fachbereich Informatik und Mathematik

LabFlow

Rollenbasierte Ausleihe von Laborgeräten

Lastenheft und technische Projektdokumentation

Modul **Large Scale IT and Cloud Computing**

Gruppenprojekt 2026

Autoren:	Zakaria Sabiri Fihi Saad Othmane Tayani
Dozent:	Prof. Thomas Eifert
Betreuung:	Dr. Marius Politze
Stand:	11. August 2026

Jülich, August 2026

1 Individueller Beitrag: Zakaria Sabiri

Verantwortete Bereiche

Gesamtarchitektur, REST API und HTTP Verträge, Azure Infrastruktur, OpenTofu, GitLab CI und CD, Object Storage, Sicherheitsintegration sowie technische Endabnahme

Mein Schwerpunkt lag auf dem technischen Rückgrat von LabFlow. Ich habe die Anwendung so strukturiert, dass Domäne, Anwendungsfälle und technische Adapter klar getrennt sind. Diese Trennung war für das Projekt wichtig, weil dieselbe Fachlogik lokal mit In Memory Repositories, im Docker Verbund mit Azurite und in der Cloud mit Azure Blob Storage funktionieren sollte. Die Repository Interfaces liegen deshalb in der Application Schicht; das Azure SDK bleibt vollständig im Storage Adapter. Dadurch konnten Fachregeln unabhängig von Cloud Diensten getestet werden.

1.1 REST API und HTTP Vertrag

Ich habe die REST Schnittstelle zwischen Frontend und Fachlogik entworfen und integriert. Die Controller bilden Geräte, Anträge, Freigaben, Übergaben, Audit Events und Authentifizierung ab. Einheitliche HTTP Statuscodes, Problem Details und Korrelationskennungen machen Fehler nachvollziehbar. Rolle, Laborzuordnung und Eigentümerschaft werden vor jedem geschützten Aufruf serverseitig geprüft.

1.2 Azure Infrastruktur mit OpenTofu

Die Azure Umgebung wurde vollständig als Code modelliert. OpenTofu erstellt Resource Group, Storage Account mit privatem Blob Container, Container Registry, Netzwerk, Subnetz, Network Security Group, statische Public IP mit DNS Namen und eine Ubuntu VM. Damit bleibt die Kursumgebung nachvollziehbar und bildet trotzdem den vollständigen Bereitstellungsweg ab. Port 80 ist öffentlich erreichbar; SSH bleibt auf einen administrativen CIDR Bereich begrenzt.

Der Storage Account verwendet lokale Redundanz, TLS 1.2, Blob Versionierung und eine Löschaufbewahrung von sieben Tagen. Fachliche Dokumente bleiben damit über Container und VM Neustarts hinweg erhalten. Die Container Registry speichert Backend und Frontend Images mit der vollständigen Commit SHA. Damit lässt sich jede laufende Revision auf einen konkreten Git Stand zurückführen.

OpenTofu verwendet GitLab Remote State mit Sperr und Entsperrpunkten. Jeder Job initialisiert ihn mit kurzlebigen Job Zugangsdaten, sodass Plan und Apply getrennt bleiben und keine dauerhaften Zugangsdaten im Repository liegen. Cloud Init installiert die Containerlaufzeit, erzeugt die Konfiguration und registriert `labflow.service`. Spätere Releases aktualisieren Images und Konfiguration, ohne die VM unnötig zu ersetzen.

1.3 Automatisierte Bereitstellung und technische Absicherung

Die GitLab Pipeline prüft Backend, Frontend, Shell Skripte und OpenTofu und erzeugt anschließend einen reproduzierbaren Plan. Nach manueller Freigabe werden Infrastruktur und serverseitig gebaute Images bereitgestellt. Der Deployment Job startet exakt die Images der aktuellen Commit SHA. Der Verify Job prüft Health Endpunkt, Authentifizierung, Frontend Asset Hash und OIDC Callback. Ein manueller Destroy Job entfernt die Umgebung kontrolliert.

Zur Endintegration gehörten außerdem GitLab OpenID Connect, serverseitige Sitzungen, CSRF Schutz, Rollenabbildung und die Übergabe geschützter Variablen. Fehler aus parallelen Blob Änderungen werden über ETags erkannt und als HTTP 409 gemeldet. Damit ist meine Arbeit nicht nur die Bereitstellung einzelner Azure Ressourcen, sondern die belastbare Verbindung von Quellcode, Tests, Identität, Persistenz und laufender Infrastruktur.

Nachweis im Repository:

`adapter.rest`, `RestExceptionHandler`, `infra`, `ci`, `.gitlab-ci.yml`, `adapter.storage`, `security` sowie die erfolgreichen Azure Pipeline Stufen von Test bis Verify.

2 Individueller Beitrag: Fihi Saad

Verantwortete Bereiche

Borrower und Lab Manager Prozess, Antragsprüfung, Zugangsvoraussetzungen, rollenbezogene Oberfläche und fachliche Tests

Ich habe den fachlichen Weg vom Ausleihwunsch bis zur Entscheidung durch die Laborleitung umgesetzt. Dazu gehört zunächst die Borrower Sicht: Ein Gerät wird aus dem Bestand gewählt, der Nutzungszeitraum und der Verwendungszweck werden erfasst und der Antrag wird zunächst als Entwurf gespeichert. Bei Geräten mit besonderem Risiko muss zusätzlich eine absolvierte Unterweisung oder Qualifikation beschrieben werden. Erst nach der ausdrücklichen Einreichung wird der Antrag für den Lab Manager sichtbar.

2.1 Prüfung und Entscheidung

Die von mir umgesetzte Freigabeansicht zeigt ausschließlich eingereichte Anträge des eigenen Labors. Für jeden Antrag werden Gerät, anfragende Person, Zeitraum, Zweck und gegebenenfalls der Qualifikationsnachweis gemeinsam dargestellt. Bei einer Genehmigung legt der Lab Manager ein verbindliches Rückgabedatum fest. Dieses Datum muss innerhalb des beantragten Zeitraums liegen. Ein eingeschränktes Gerät kann nur genehmigt werden, wenn die Zugangsvoraussetzung ausdrücklich bestätigt wurde. Die Bestätigung wird mit Name und Zeitpunkt als **AccessVerification** im Antrag gespeichert.

Eine Genehmigung ändert den Antrag auf **APPROVED** und reserviert das Gerät. Vorher wird erneut geprüft, ob es noch verfügbar ist. Damit kann ein veralteter Browserstand keine Doppelvergabe erzeugen. Eine Ablehnung erfordert einen nachvollziehbaren Grund und setzt den Antrag auf **REJECTED**. Rolle und Laborkontext werden nicht nur in der Oberfläche, sondern im Backend geprüft. Ein Lab Manager aus einem anderen Labor erhält deshalb unabhängig von der dargestellten Navigation HTTP 403.

2.2 Oberfläche und Qualitätssicherung

Ich habe die zugehörigen React Bereiche für die Antragserstellung, die Übersicht eigener Vorgänge und die Freigabewarteschlange ausgearbeitet. Ladezustände, leere Listen, Validierungsfehler und erfolgreiche Entscheidungen werden verständlich dargestellt. Unzulässige Aktionen erscheinen nicht in der rollenabhängigen Navigation. Die Oberfläche bleibt dabei nur eine Benutzerführung; die verbindliche Entscheidung fällt immer im Application Service.

In meinem Arbeitspaket habe ich die Statuswechsel **DRAFT** nach **SUBMITTED**, **SUBMITTED** nach **APPROVED** oder **REJECTED** sowie die Stornierung vor der Ausgabe geprüft. Zusätzlich decken die Tests Eigentümerschaft, Laborkontext, Rückgabedatum, Qualifikationsnachweis und die erneute Verfügbarkeitsprüfung ab. Der vollständige REST Integrationstest verbindet diese Regeln mit den erwarteten HTTP Statuscodes und AuditEvents.

2.3 Abgrenzung und Schnittstellen

Mein Arbeitspaket endet mit einem genehmigten und reservierten Gerät. Die physische Ausgabe bleibt bewusst beim Technician. Diese Trennung entspricht dem Vier Augen Prinzip: Die Laborleitung trifft die fachliche Entscheidung, während eine andere Rolle Identität, Zubehör und Gerätezustand bei der Übergabe dokumentiert. Die Übergabe an den Technician erfolgt über den stabilen Status **APPROVED** und die gemeinsame **LoanRequest** Entität.

Nachweis im Repository:

RequestsPage, **ApprovalsPage**, **LoanRequestController**, **ApprovalController** sowie die Genehmigungs und Autorisierungstests.

3 Individueller Beitrag: Othmane Tayani

Verantwortete Bereiche

Gerätebestand, Bilder und Zugangsklassen, Ausgabe und Rückgabe, Zustandsdokumentation sowie Technician Oberfläche

Ich habe die technischen Vorgänge am realen Gerät umgesetzt. Mein Bereich beginnt bei der Bestandserfassung durch einen Technician. Ein neues Gerät benötigt Name, Typ, Inventarnummer, Labor, Zugangsklasse und ein Bild. Die unterstützten Typen sind nicht auf IT Geräte begrenzt. Neben Laptops und Sensoren können Messgeräte, Elektrowerkzeuge, Löttechnik, Laborgeräte, Kameras und weitere Gerätetypen erfasst werden.

3.1 Sichere Bestandserfassung

Bei der Bestandserfassung habe ich die Zugangsklasse als fachlich relevante Eigenschaft berücksichtigt. `OPEN` beschreibt den Standardzugang. `INSTRUCTION_REQUIRED` verlangt eine Unterweisung. `QUALIFICATION_REQUIRED` verlangt einen formalen Nachweis. Für beide eingeschränkten Klassen muss eine konkrete Voraussetzung angegeben werden. Diese Information wird später im Antrag und in der Freigabeansicht sichtbar. Damit entscheidet nicht allein die technische Verfügbarkeit darüber, ob ein Gerät ausgeliehen werden darf.

Ich habe Gerätebilder als Multipart Upload angebunden. Der Server akzeptiert ausschließlich JPEG, PNG oder WebP bis vier Megabyte, prüft Dateisignatur und Abmessungen und speichert das Bild in einem getrennten Blob Prefix. Die Inventarnummer muss innerhalb des Labors eindeutig sein. Wird das JSON Dokument nach einem erfolgreichen Bildupload nicht gespeichert, entfernt der Service das Bild wieder. Dadurch entstehen keine verwaisten Dateien.

3.2 Ausgabe und Rückgabe

Die von mir entwickelte Technician Warteschlange trennt geplante Ausgaben und erwartete Rückgaben. Bei einer Ausgabe werden anfragende Person, Gerät, Termin und Zubehör geprüft. Der Technician dokumentiert den Ausgangszustand und optionale Bemerkungen. Die API akzeptiert den Vorgang nur, wenn der Antrag `APPROVED`, das Gerät `RESERVED` und noch kein `CheckoutRecord` vorhanden ist. Bei eingeschränkten Geräten muss außerdem die Bestätigung der Zugangsvoraussetzung gespeichert sein. Danach stehen Antrag und Gerät auf `CHECKED_OUT`.

Bei der Rückgabe erfasst der Technician Zeitpunkt, Zustand und Bemerkungen. `FAULTLESS` und `MINOR_WEAR` geben das Gerät wieder als `AVAILABLE` frei. `REVIEW_REQUIRED` setzt es dagegen auf `MAINTENANCE`, sodass keine neue Anfrage möglich ist, bevor eine fachliche Prüfung erfolgt. Der `CheckoutRecord` enthält sowohl Ausgabe als auch Rückgabe und bewahrt die verantwortlichen Personen und Zeitpunkte dauerhaft auf.

3.3 Tests und Rollenabgrenzung

Mit meinen Tests habe ich Bildvalidierung, eindeutige Inventarnummern, reservierte Geräte, doppelte Ausgaben, vollständige Rückgabedaten und den Wechsel in den Wartungsstatus geprüft. Ein Technician darf weder Anträge genehmigen noch fremde Laborvorgänge bearbeiten. Umgekehrt können Borrower und Lab Manager keine Ausgabe oder Rückgabe buchen. So bleibt die Verantwortung für den physischen Gerätefluss eindeutig.

Nachweis im Repository:

`CreateEquipmentForm`, `HandoverPage`, `EquipmentService`, `HandoverController`, `CheckoutRecord` und die zugehörigen Tests.

4 Geschäftsprozess, Rollen und Entitäten

LabFlow bildet eine Geräteausleihe von der Auswahl bis zur bestätigten Rückgabe ohne Medienbruch ab. Der Prozess beginnt bei einem verfügbaren Gerät. Der Borrower erstellt einen Entwurf und reicht ihn ein. Der Lab Manager prüft Zweck, Zeitraum und Zugangsvoraussetzung. Nach der Genehmigung reserviert das System das Gerät. Der Technician dokumentiert Ausgabe und Rückgabe. Jeder Statuswechsel erzeugt ein unveränderliches AuditEvent.

4.1 Rollen

Rolle	Erlaubte Aktionen	Fachliche Grenze
Borrower	Geräte lesen, eigenen Antrag erstellen, einreichen, lesen und vor Ausgabe stornieren	Nur eigene Anträge im zugewiesenen Labor
Lab Manager	Eingereichte Anträge prüfen, genehmigen oder begründet ablehnen	Nur Anträge des eigenen Labors, keine physische Übergabe
Technician	Geräte erfassen, genehmigte Geräte ausgeben und Rückgaben dokumentieren	Nur Vorgänge des eigenen Labors, keine Entscheidung über Anträge

Verbindliche Geschäftsregeln

Ein Antrag gehört genau zu einem Gerät, einem Borrower und einem Labor. Statussprünge außerhalb des definierten Ablaufs werden mit HTTP 409 abgelehnt. Genehmigung und Ausgabe prüfen die Verfügbarkeit erneut. Eine Rückgabe setzt ein Gerät nur dann wieder frei, wenn kein Prüfbedarf dokumentiert wurde. Rolle, Labor und Eigentümerschaft werden bei jeder Aktion serverseitig geprüft.

Fachliche Kernobjekte

Objekt	Bedeutung im Prozess
Equipment	Laborgerät mit Inventarnummer, Typ, Zustand, Zugangsregel und Verfügbarkeit
LoanRequest	Ausleihantrag mit Zeitraum, Zweck, Status, Borrower, Labor und Qualifikationsnachweis
CheckoutRecord	Nachweis über Ausgabe, Rückgabe, beteiligten Technician und dokumentierten Gerätezustand
AuditEvent	Unveränderlicher Nachweis jedes fachlich relevanten Statuswechsels
AccessVerification	Bestätigung, dass Unterweisung oder Qualifikation vor der Freigabe geprüft wurde

4.2 Anwendungsfälle

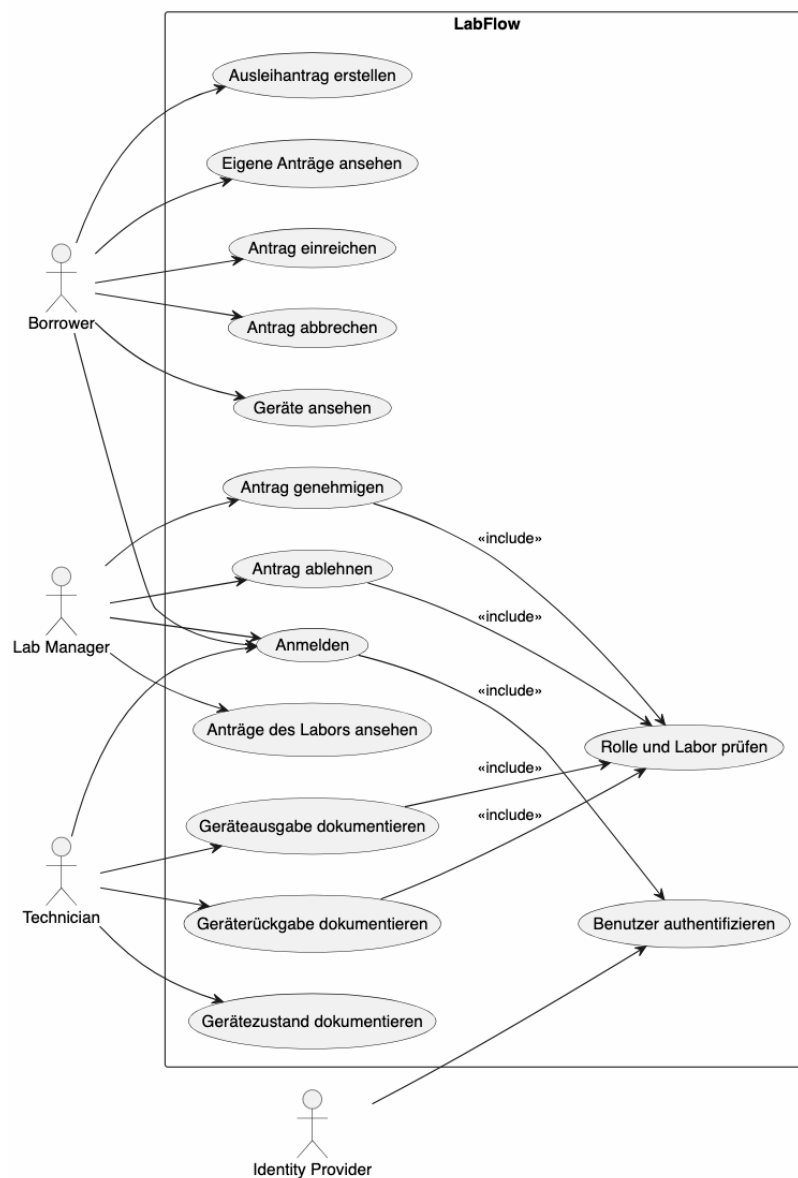


Abbildung 1: Anwendungsfälle aus dem LabFlow Lastenheft

Das Diagramm zeigt die fachlichen Aktionen der drei Rollen und die beiden gemeinsamen Sicherheitsfunktionen. Authentifizierung sowie die Prüfung von Rolle und Labor sind keine optionalen Oberflächenfunktionen, sondern Voraussetzungen für jeden geschützten Anwendungsfall.

4.3 Fachliches Klassenmodell

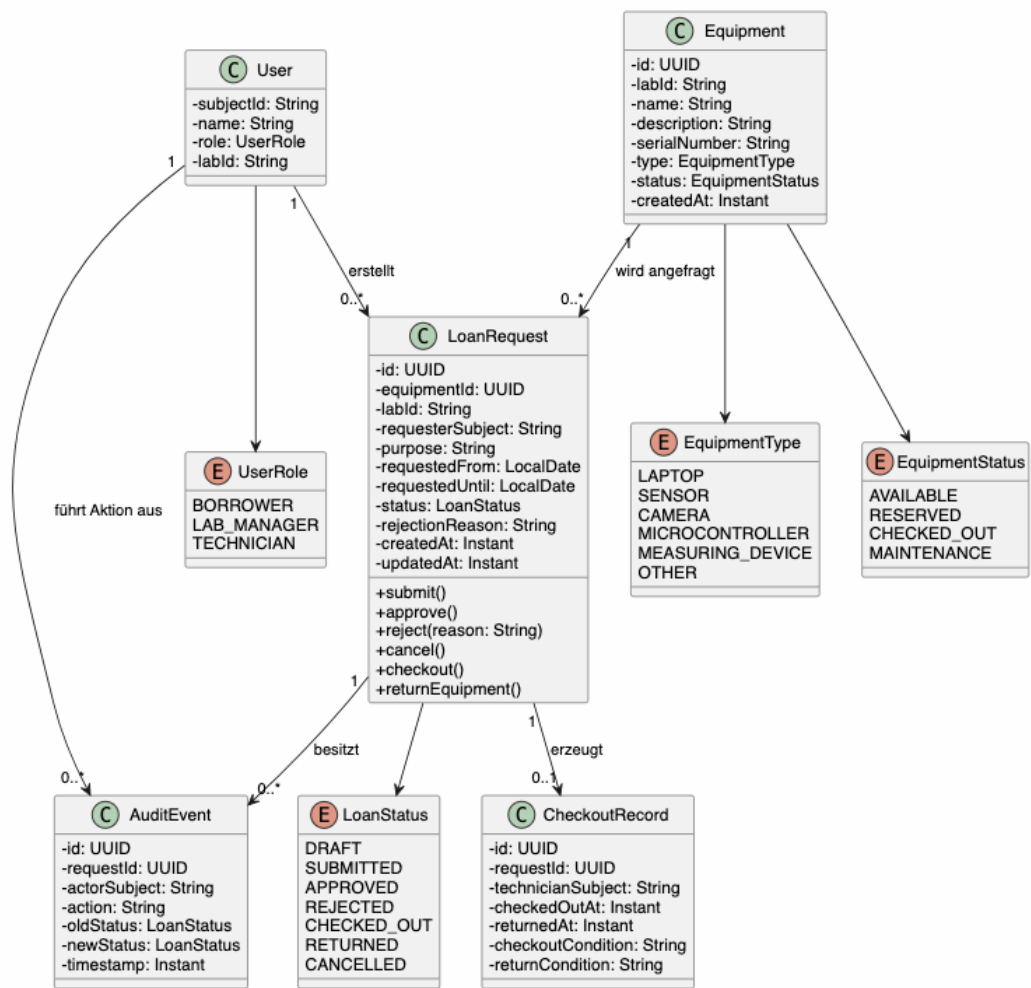


Abbildung 2: Fachliches Klassenmodell aus dem LabFlow Lastenheft

Das Diagramm dokumentiert das Grobmodell aus dem Lastenheft. Die Implementierung speichert keine eigene Benutzerklasse. Stattdessen stellt die Sitzung für jeden Aufruf einen `AuthenticatedActor` bereit. `LoanRequest` enthält außerdem die bestätigte `AccessVerification`. Die übrigen Beziehungen bleiben unverändert.

5 Ablaufdiagramm mit API Methoden und Rollen

Der folgende Ablauf wurde aus dem LabFlow Lastenheft übernommen. Er verbindet jeden fachlichen Hauptschritt mit der ausführenden Rolle und der schreibenden REST Methode.

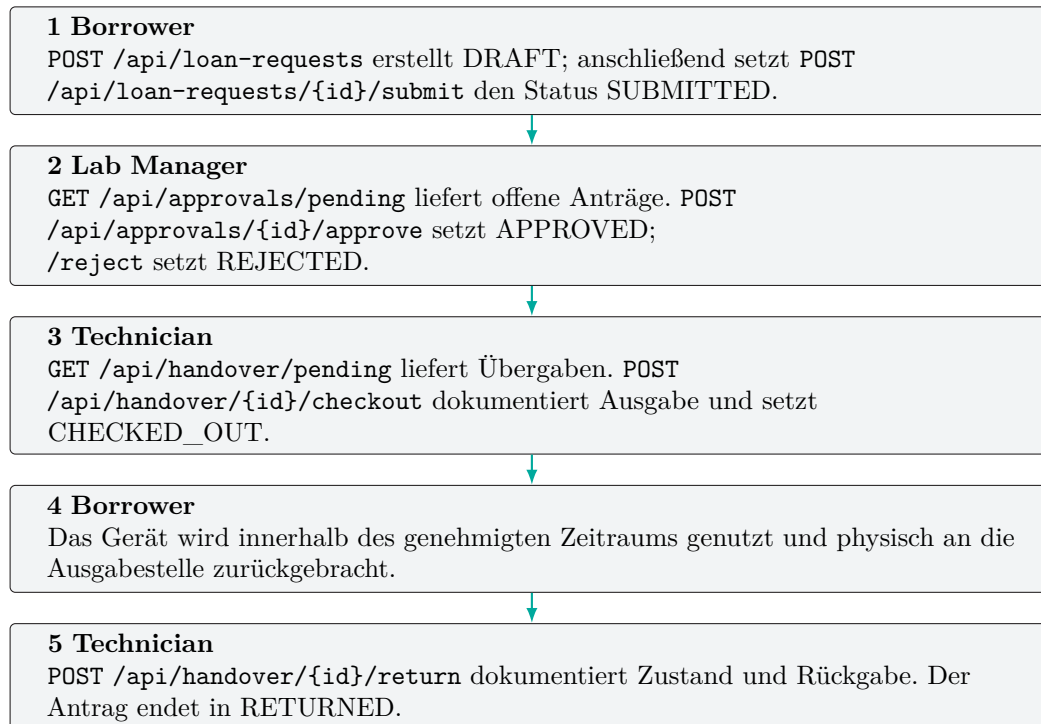


Abbildung 3: Prozessablauf mit REST API und Rollen aus dem Lastenheft

5.1 Weitere Endpunkte und HTTP Semantik

Methode	Pfad	Berechtigung und Zweck
GET	/api/equipment	alle Rollen, Geräte des eigenen Labors
GET	/api/loan-requests	Borrower, eigene Anträge
GET	/api/loan-requests/{id}	Borrower, eigener Einzelvorgang
POST	/api/loan-requests/{id}/cancel	Borrower, vor Ausgabe stornieren
GET	/api/audit-events	Lab Manager und Technician, Audit Trail des Labors
GET	/api/auth/csrf	CSRF Token für schreibende Anfragen

Erstellen liefert HTTP 201. Ungültige Eingaben liefern 400, fehlende Authentifizierung 401, fehlende Berechtigung 403, unbekannte Objekte 404 und Status oder Versionskonflikte 409. Fehler verwenden einheitliche Problem Details mit Code und Korrelationskennung. Jeder erfolgreiche Prozessschritt speichert zusätzlich ein AuditEvent. Vor Genehmigung und Ausgabe werden Berechtigung, Zustand und Verfügbarkeit erneut serverseitig geprüft.

6 Persistenzschicht und Repositories

Die Persistenz folgt dem Repository Muster. `EquipmentService` und `LoanRequestService` arbeiten ausschließlich mit Interfaces aus der Application Schicht. Für schnelle isolierte Tests existieren In Memory Implementierungen. Im lokalen Docker Verbund und in Azure werden die Interfaces durch Azure Adapter umgesetzt. Dadurch bleibt die Fachlogik unabhängig vom Speicherort.

Ebene	Baustein	Verantwortung
Application	Repository Interfaces	Speicherunabhängige Ports für Geräte, Anträge, Übergaben, Auditereignisse und Bilder
Test	In Memory Repositories	Schnelle, isolierte Ausführung der Fach und REST Tests
Lokal	Azure Adapter mit Azurite	Persistenter Entwicklungsbetrieb über dieselbe Blob API wie in Azure
Produktion	Azure Adapter mit Blob Storage	JSON Dokumente, Binärbilder, ETags, Versionierung und Löschaufbewahrung

6.1 Blob Prefixes

Dokumentart	Blob Name
Gerät	<code>labs/{labId}/equipment/{equipmentId}.json</code>
Gerätebild	<code>labs/{labId}/equipment-images/{equipmentId}.bin</code>
Ausleihantrag	<code>labs/{labId}/loan-requests/{requestId}.json</code>
Ausgabe und Rückgabe	<code>labs/{labId}/checkout-records/{requestId}.json</code>
Auditereignis	<code>labs/{labId}/audit-events/{epochMillis}-{eventId}.json</code>

Das erste Segment trennt Daten nach Labor. Das zweite Segment trennt die fachlichen Dokumentarten. Änderbare Objekte besitzen einen stabilen Namen aus ihrer UUID. AuditEvents tragen Zeitpunkt und UUID im Namen, weil sie ausschließlich ergänzt und nie aktualisiert werden. Bilder liegen getrennt von den JSON Metadaten, damit Inhaltstyp und Binärdaten ohne Base64 Overhead gespeichert werden.

6.2 Serialisierung und Parallelität

Jackson serialisiert Java Records als UTF 8 JSON. UUID, Datum und Zeitpunkt werden in ihren standardisierten Textformen gespeichert. Jedes änderbare Dokument besitzt eine numerische **revision**. Beim ersten Speichern verwendet der Adapter **If-None-Match: ***. Bei einer Aktualisierung muss die Revision genau um eins steigen und **If-Match** muss dem aktuellen Blob ETag entsprechen. Azure Antworten 409 oder 412 werden als **ConcurrencyConflictException** auf HTTP 409 abgebildet.

Dieses Verfahren verhindert verlorene Änderungen: Wenn zwei Anfragen dieselbe Revision lesen, kann nur die erste erfolgreich schreiben. Die zweite muss den aktuellen Zustand neu laden, statt die fremde Änderung unbemerkt zu überschreiben. AuditEvents und neue Bilder werden immer mit **If-None-Match: *** angelegt und sind damit unveränderlich.

6.3 Lokaler und produktiver Betrieb

Das Memory Profil hält Daten nur für isolierte Tests. Docker Compose verwendet Azurite mit einem persistenten Volume und denselben Azure SDK Aufrufen wie die Cloud. In Azure liegt der Container privat in einem Storage Account mit Blob Versionierung und Löschaufbewahrung. Secrets und Connection Strings stehen nicht im Git Repository, sondern werden zur Laufzeit aus geschützten CI Variablen und OpenTofu Ausgaben erzeugt.

7 Architektur, Infrastruktur und Deployment

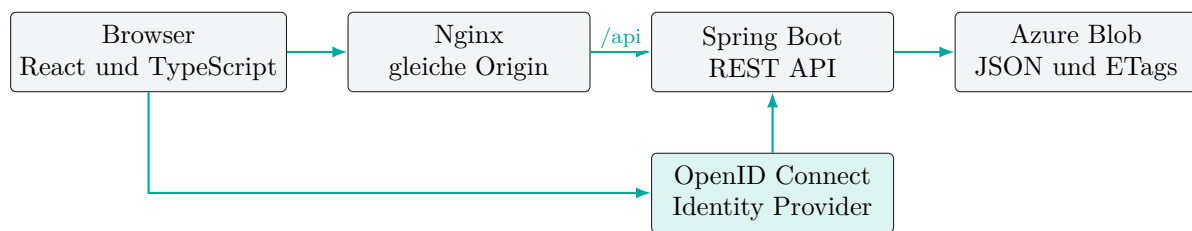


Abbildung 4: Laufzeitarchitektur aus dem LabFlow Lastenheft

7.1 Verwendete Container

Container	Umgebung	Aufgabe
frontend	Lokal und Azure	React Produktionsbuild, statische Dateien, Reverse Proxy und /healthz
backend	Lokal und Azure	Java REST API, Fachlogik, Sitzung, OIDC und Azure Repositories
keycloak	Nur lokal	OpenID Connect Provider mit drei Testrollen
azurite	Nur lokal	Emulator für Azure Blob Storage mit persistentem Volume

7.2 Azure Infrastrukturkomponenten

OpenTofu verwaltet Resource Group, Storage Account und privaten Blob Container, Azure Container Registry, Virtual Network, Subnet, Network Security Group, statische Public IP mit DNS, Netzwerkschnittstelle und Ubuntu VM. Die VM führt nur Frontend und Backend aus. Nginx veröffentlicht Port 80; Spring Boot bleibt im internen Compose Netzwerk. Cloud Init installiert die Containerlaufzeit und richtet den Systemd Dienst ein.

7.3 Automatisierungsschritte

Stufe	Automatisierter Nachweis
Test	Maven Verify, Vitest, TypeScript, Shell Tests und OpenTofu Validate
Plan	Reproduzierbarer OpenTofu Plan mit GitLab Remote State und Sperre
Provision	Manuell freigegebenes OpenTofu Apply für die Azure Ressourcen
Image	Serverseitiger ACR Build mit unveränderlicher Commit SHA
Deploy	Start genau dieser Revision über Azure Run Command
Verify	Health, Auth Konfiguration, Frontend Asset Hash und OIDC Callback
Teardown	Manuelles und kontrolliertes Entfernen der verwalteten Ressourcen

Damit sind die drei bewerteten Ebenen miteinander verbunden: Die Java Implementierung bildet den Geschäftsprozess als REST API ab, die Persistenz speichert versionierte Dokumente in Object Storage und die GitLab Pipeline liefert den getesteten Stand reproduzierbar in die Azure Cloud aus.

Abnahmepunkt	Technischer Nachweis
Lauffähige Java REST API	Maven Verify und erfolgreicher Workflow Test
Persistentes Repository	Azure Blob Prefixes, JSON und ETag Konfliktschutz
SSO und mindestens zwei Rollen	OIDC Integration und drei serverseitig geprüfte Rollen
Infrastructure as Code	OpenTofu Plan und Apply mit Remote State
Docker und GitLab CI	Versionierte Images, Azure Deployment und Verify Job